

Progress & Learning Report, Sprint 3

FISH GAME

SAMUEL KORÁB (24072990)

Week 1

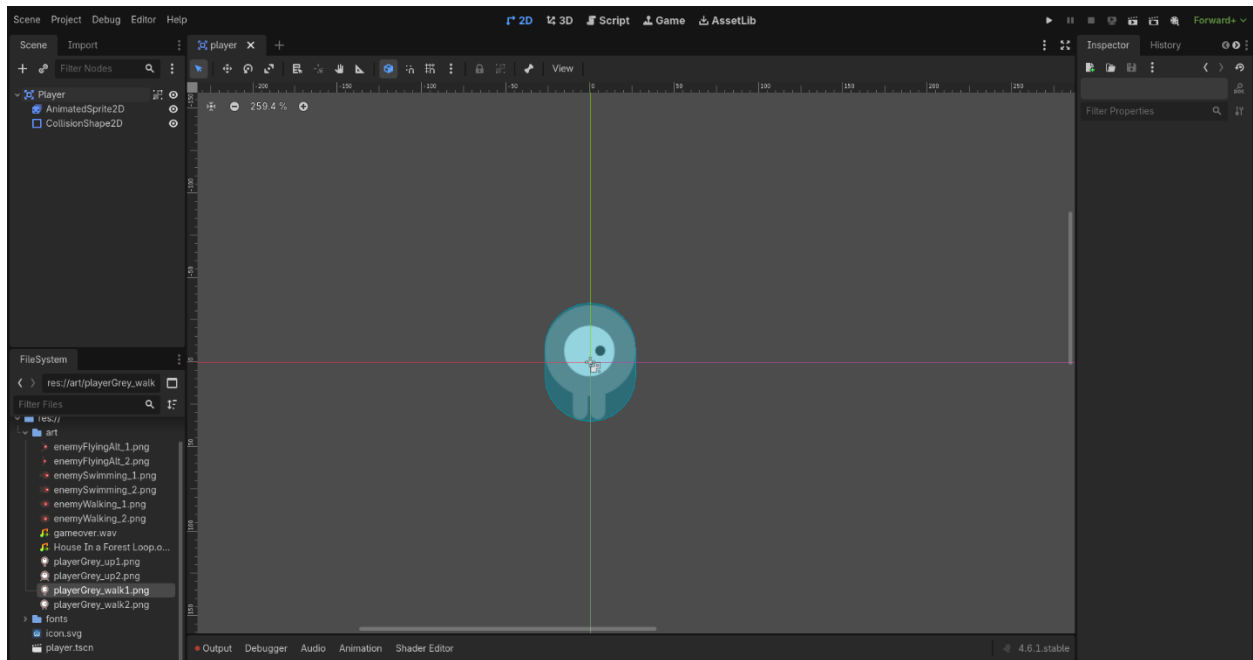
I started this sprint off by researching the options I have when it comes to engines. The whole reason I chose to do this for this split was that I heard a lot of good things about Godot, but I hadn't done the deep dive myself to see what the hype is really about.

I've also always wanted to make a fishing game. Fishing is my favorite thing in any game that has it and it just provides such a relaxing experience that I can't get anywhere else. I have a few ideas for unique and interesting fishing mechanics, such as a rhythm based one, or an aim-based clicking one inspired by *osu!*, but I don't want to box myself into design decisions this early on so I'm keeping an open mind. Basically, I'm starting off with some ideas in mind for what I want, but nothing concrete.

I watched YouTube videos, browsed Reddit threads, and even talked to AI to try to get a complete picture of the current game dev landscape. Based on the upsides and downside and the 3D project I had in mind for this sprint, I downloaded Unreal and tried it out by following a tutorial. Unfortunately, I found it quite clunky to work with, and I am not a fan of Epic Games or the corporate gaming landscape, so I immediately looked for other options. I read that while Godot might be less popular than Unity or Unreal, it has a very intuitive learning curve letting you to build up your own systems layering them on top of each other, rather than importing tons of libraries, which is the preferred way I like to work anyway.

What I found interesting is how AI was heavily pushing me down a Three.JS web-oriented path once again, presumably due to having the most training data for that. I considered it, even testing it a little, but the prospect of spending another 4 weeks of my life on JavaScript seemed rather grim, so I decided to go with Godot after all, mainly because of its open-source nature and overwhelmingly positive feedback from its users.

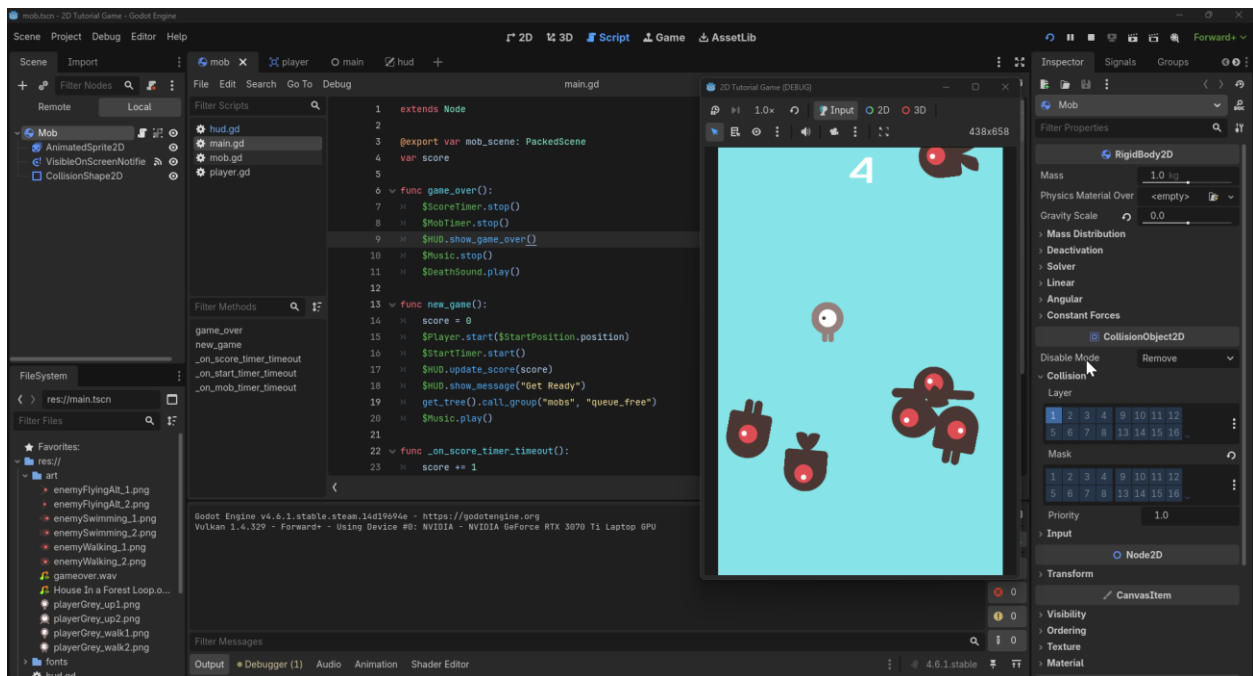
I'm a very inattentive person so tutorials are always a drag for me, I usually prefer hands-on courses like the one I did for React in sprint 1, but the Godot docs are very well written and kept up-to-date, which to me seemed like a signal to just open up the engine and see what I can make. After an entire evening of no meaningful progress, I decided to concede that web dev doesn't really translate at all into game dev, and started the "Your first 2D game" tutorial from the docs, which walks you through making a simple game called Dodge the Creeps.



Godot is unique in the fact that it allows you to use their built in, GDScript scripting language in any combination with C#, maintaining the best of both worlds for both accessibility and scalability. I really like this capability, even not having any C# experience, because it feels like the code I write now in GDScript will not go to waste if I do get to scale it in the future.

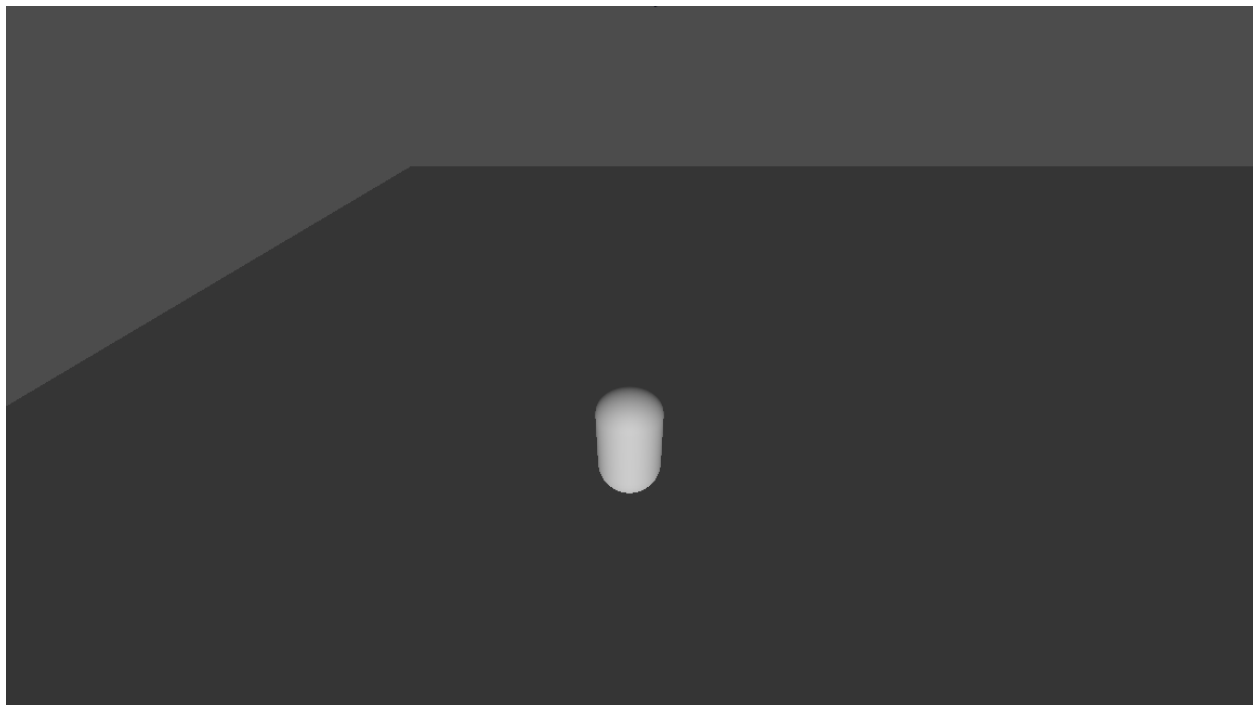
The tutorial covers the basics such as setting up the player scene, area, movement logic and signals, which are just event emitters other parts of the code can use. When a player's Area2D detects a collision, it emits a signal that other nodes can listen for, similar to how you'd use callbacks in React. You can wire everything up in the editor without the nodes needing to know about each other, which makes for very elegant code.

Coming from where everything is components and props, the node system felt surprisingly familiar. I like to think of scenes like reusable components, and the way child nodes inherit from parents reminded me of how DOM elements work. On the other hand, the way that Godot keeps references of things is still confusing to me, as there seem to be an effectively infinite methods of wiring things together, all requiring seemingly one specific configuration to function, which I have no way to know without looking it up.



Once I was done with the tutorial I felt like I had the fundamentals down, so I started trying out 3D. I followed the first few steps of the "Your first 3D game" tutorial, but after doing the basic character and space set up, I got bored and I felt like I had enough of a clue to start working on my own project.

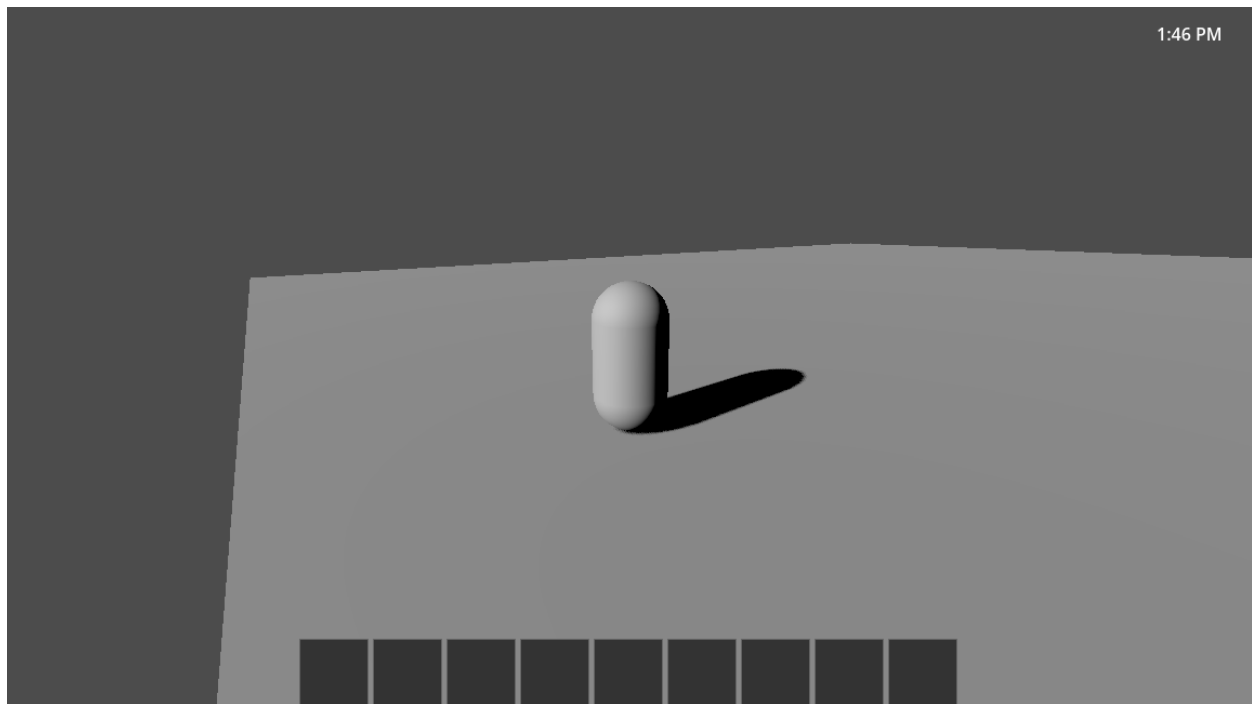
As a personal challenge, I diverted from the tutorial and made my own camera system, using a SpringArm3D node that rotates with mouse input on both axes. I also made my own movement system based on the 3D game tutorial, but adapted it to use my camera pivot as a turning point.



At this point I thought I'd just set up some basic world systems so that I can start working on the fishing features. Needing light in the scene, I thought it would be fun to try my hand at writing a time script without a tutorial.

My implementation revolves around an accumulating delta time scaled by a configurable scale variable, emitting `hour_changed` and `minute_changed` signals when those values tick over. I set up a simple HUD with just a text display, which listens to these and displays the time in 12-hour AM/PM format. A `DirectionalLight3D` rotates based on the current hour to simulate the sun. Because it's signal-based, anything in the game can listen for time changes without the clock needing to know about them. It was a nice way to try making a custom system without any tutorials, giving me some confidence to try more complex things.

My next goal was an inventory system so that I have a foundation to build the fishing mechanics on top of. I needed a way to store items for a particular player, so I set up an array that then the HUD reads to display item slots. I initially just used a `String` array, which means I can add new fish or tweak existing ones without ever touching the code. At this point I'm conditioned by TypeScript to be very mindful of my data structures, which now I plan in advance before I even write a line of code. As GDScript also supports static typing, it becomes rewarding to establish data-driven systems rather than hard-coded, the idea being that it will save on development time later. So, I begun setting up an organized data structure, creating a `Fish` resource class that stores properties like name, texture, weight range, and rarity, allowing me to tweak values in the editor.

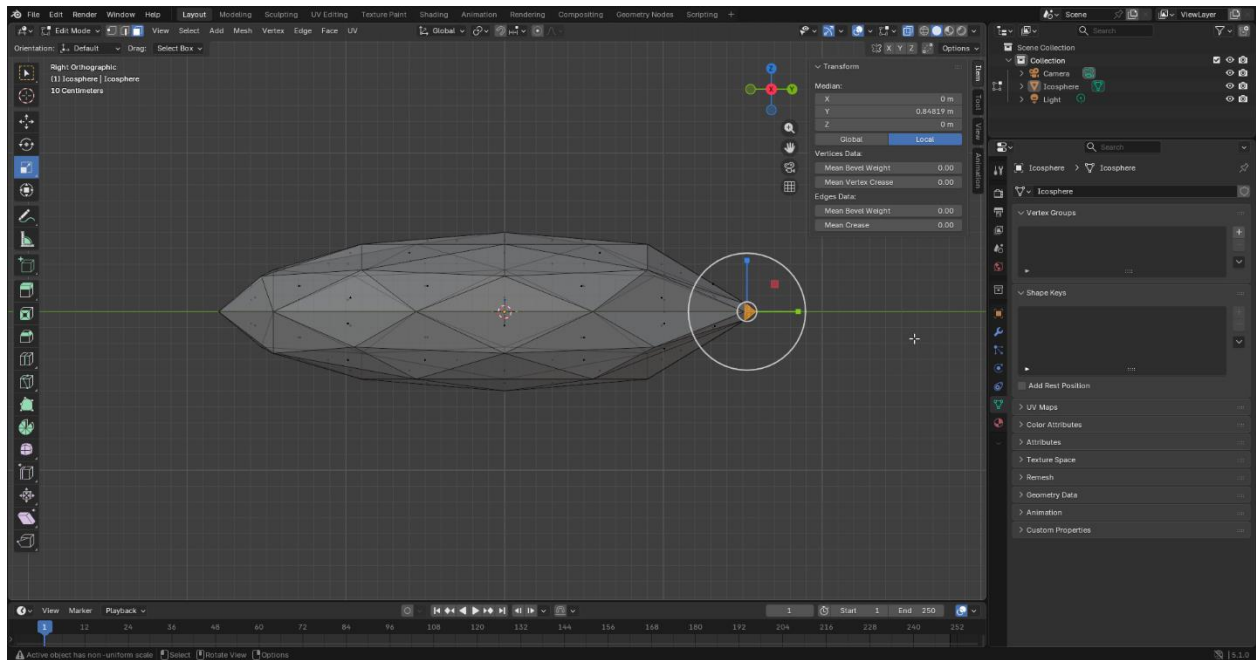


As week 1 was ending I realized game dev is a more work than I expected. I had originally planned to have a somewhat functional and playable game at the end of this sprint, but a week has passed and I had barely created a basic scene. I'm learning a ton with every little thing I add or tweak, and every iteration genuinely feels like a learning moment where I can clearly see the benefits of one solution versus the other, but that also makes it a slow and excruciating learning experience. There are a lot of things to account for in game

dev, systems have to be tailored and specialized to your game, and the learning curve gets steep really fast as you scale your project. Many people recommend working on many smaller projects rather than one big one to learn a more diverse set of skills faster, but I cannot make myself motivated to work on smaller projects, so I'm sticking to this approach even if it might not be ideal.

Week 2

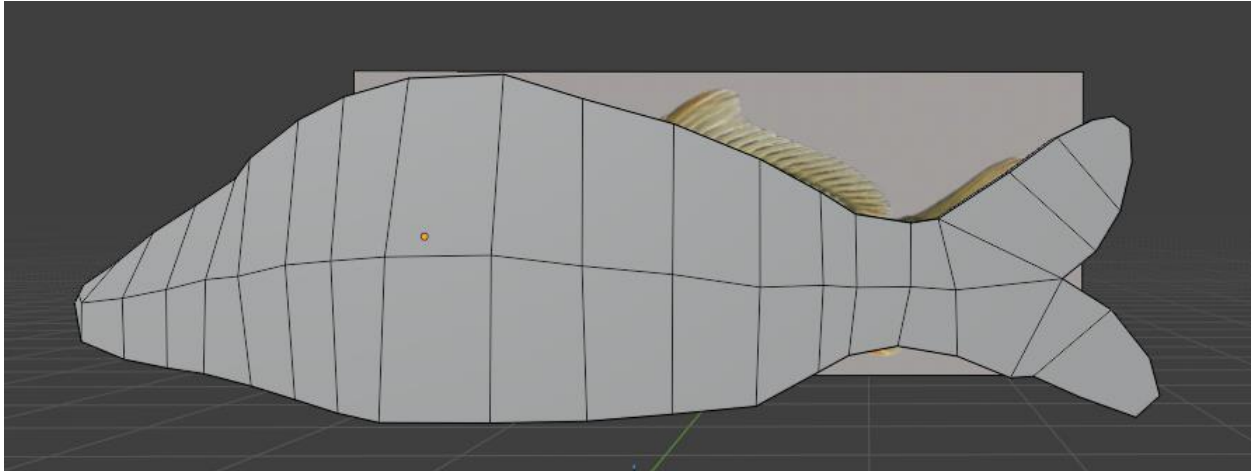
I decided it was time to try my hand at some 3D modelling, as that was another thing I wanted to pick up during this course. I started up Blender for the first time in my life and played around with it without a tutorial. It genuinely took me 10 minutes to get used to the controls to move the camera and set up basic shapes. I made this fish looking object and while I did like it, I thought there is absolutely no way I will figure out how to turn it into anything resembling a fish by myself, so I looked up some tutorials and just watched them to see how people do things.



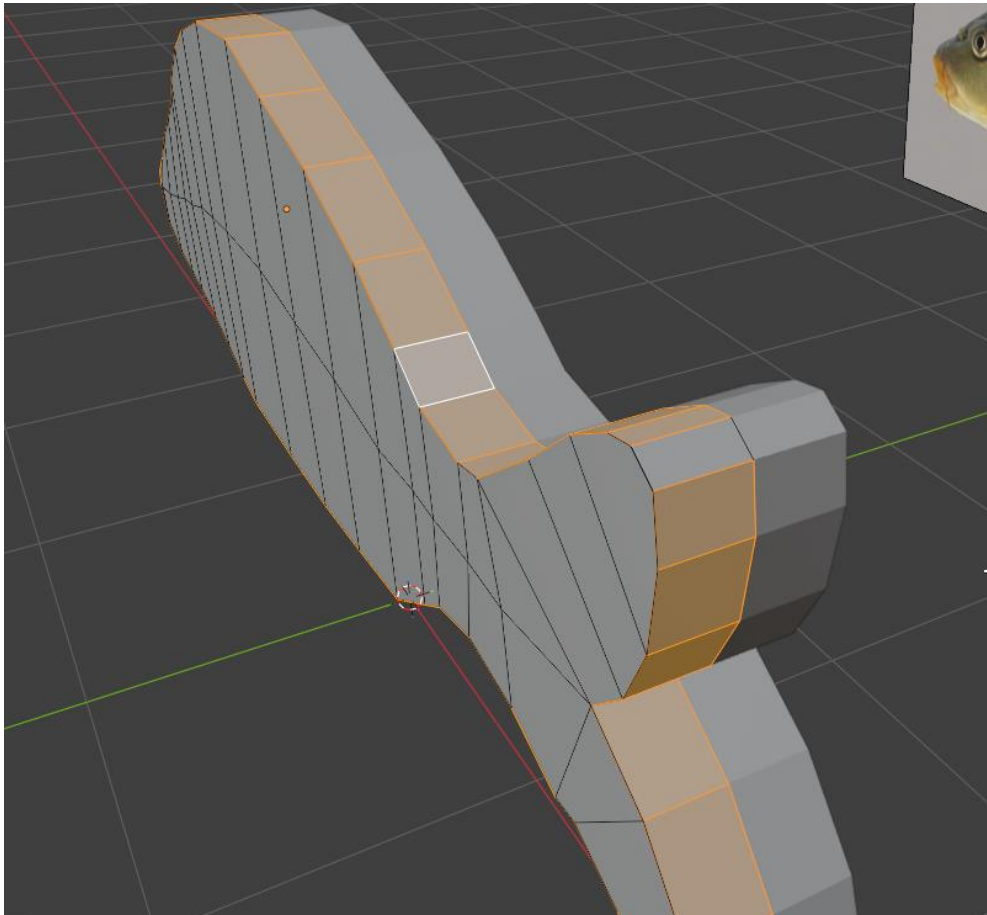
I learned that having good reference images helps a lot, so I found this beautiful specimen on Google Images and started working on getting the shape right. It was a headache to work with Blender at first, mainly because I had to Google even the most basic things.



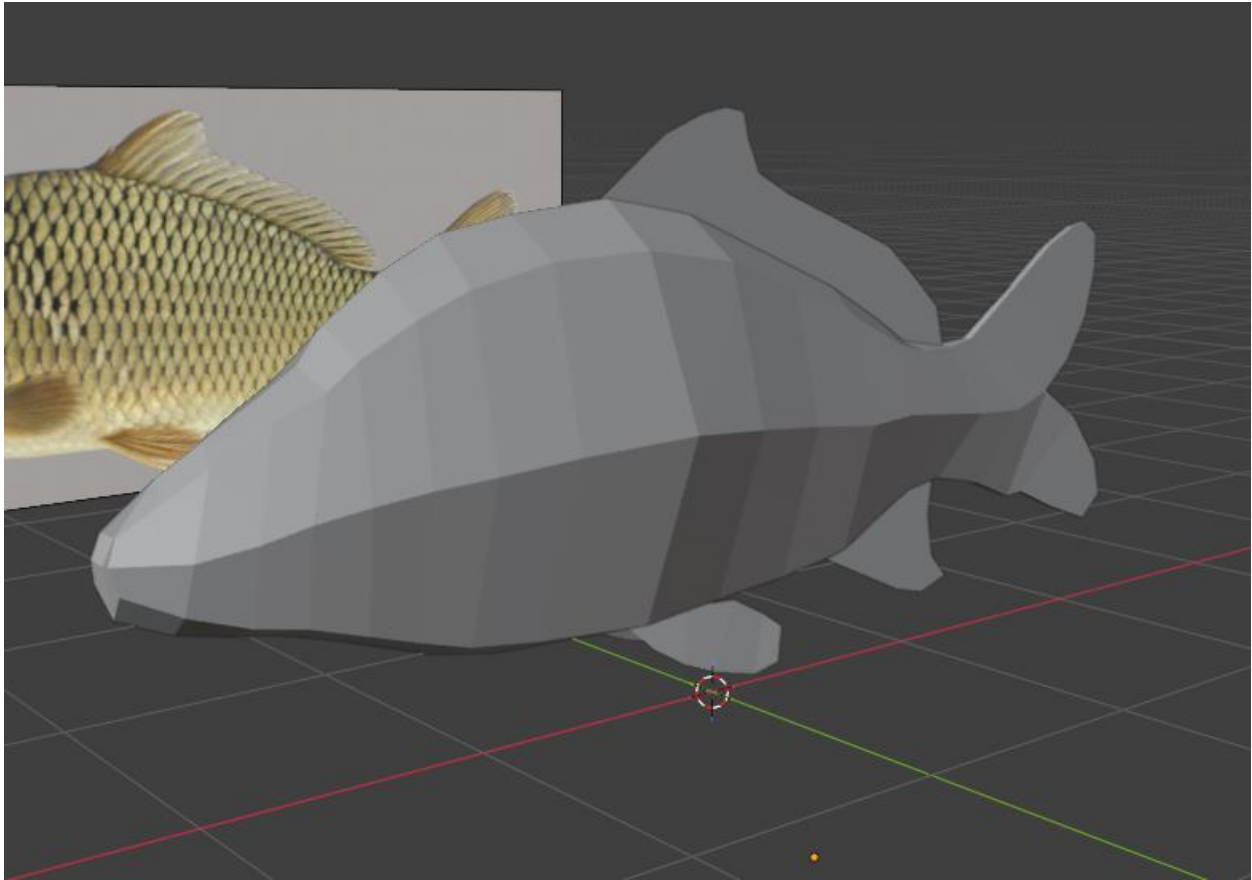
I first started with a flat plane and used CTRL + R to split the vertices to create new polygons manually, tracing the shape of the fish. Afterwards, I expanded the flat plane to add depth, then I looked into the ways that I could make this easy for myself, since the fish is symmetrical, I thought I could just mirror it later. Turns out I could just use mirror modifier which renders a mirrored copy on the other side, so making changes on one side automatically makes them on the other.



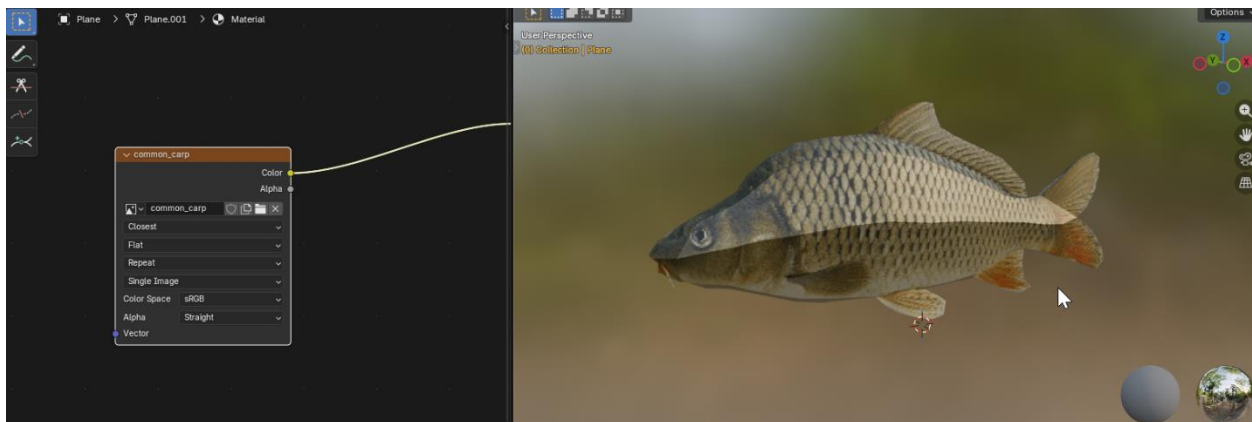
I selected the newly expanded side of the model and used the scale tool on the Y axis to pinch it and give it a rounder shape. I also selected the fin separately and scaled that even more to make it thinner.



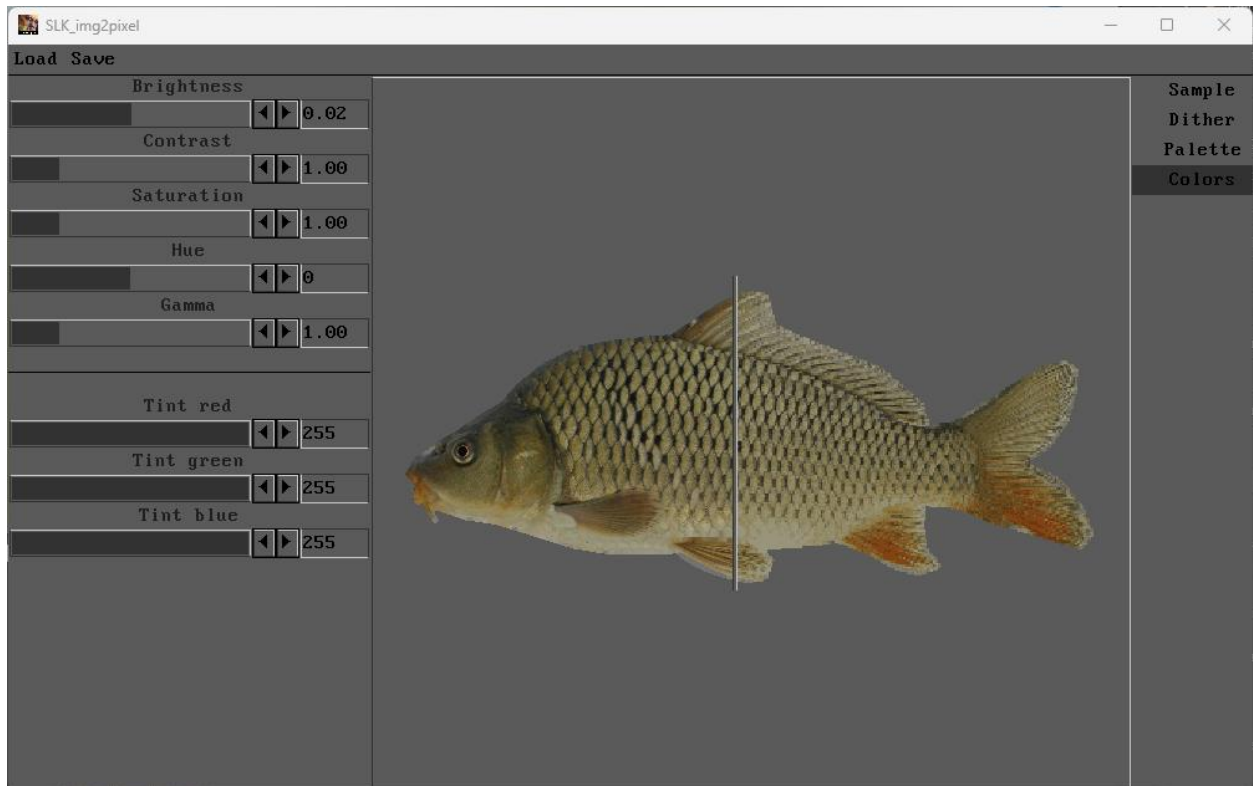
Subsequently I traced the fins with more 2D planes and created the fins, leaving me with this as a result:



I was very happy with how it looked, especially for my first time using Blender, but I still felt like I needed to polish it a little. I wanted to try my hand at texture mapping, so I found a 1.5-minute YouTube short explaining it and just decided to try my hand at it. My reference image was good enough for a texture map, so I just used that. Texture mapping itself was just lining up the model over the image and ended up being surprisingly easy and intuitive in Blender. The modelling took me about 6 hours, while the texture mapping was done in less than 30 minutes or so, resulting in this:



While I did like the look of the model already, I've had a clear artstyle in mind for my game, inspired by indie games such as Webfishing and Lethal Company. These games use a PS1-style shader with extremely simple textures and low-poly models to achieve a unique retro look that I really like. I looked up how people achieve pixelated textures for low-poly models, and found a great tool called Img2Pixel which can pixelate images with many configurable options to the sampling methods, dithering, color palette, and more. I looked into how I could export my fish reference image as a texture, since it still was using the original image as the texture. I learned about packing textures in Blender, which removes any areas not UV mapped from the source image, allowing me to export it cleanly. I then put this into Img2Pixel and played around with it until I got the look I wanted.

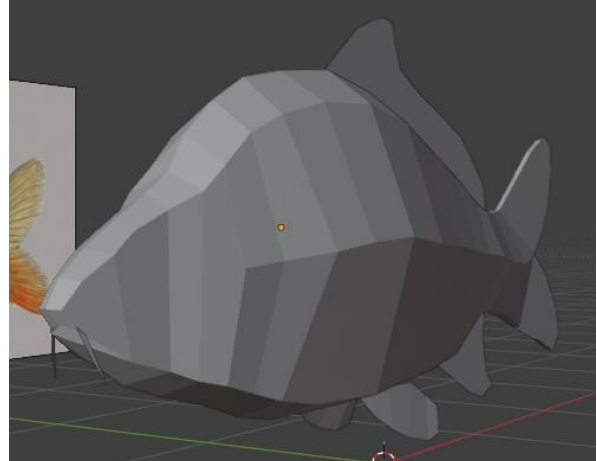
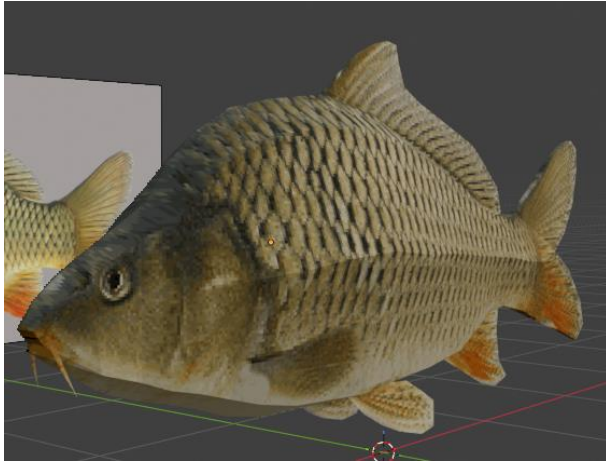


I then imported it back into Blender, but it was unfortunately really blurry. I looked into how this could be solved, and I found that Blender uses linear interpolation by default. Changing it to "Closest" instead gave it the pixelated look I was going for.

Before/After:

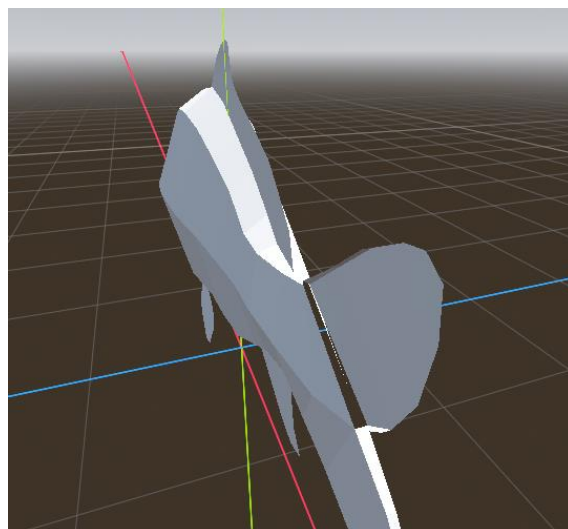


With the texture finished, I went back to the model and polished it, adding an extra 2D plane for the fangs that the carp has, mapping the texture onto them. I also flattened the head so that the line going through the entire fish stops at the head, since that part is flatter on fish IRL. The result of it was this guy, who I was really pleased with at this point. I learned a ton in the process of this, and even though from start to finish it took an excruciating 8 hours, I got a lot faster as I got used to the hotkeys, so now the same process would probably take me half the time.



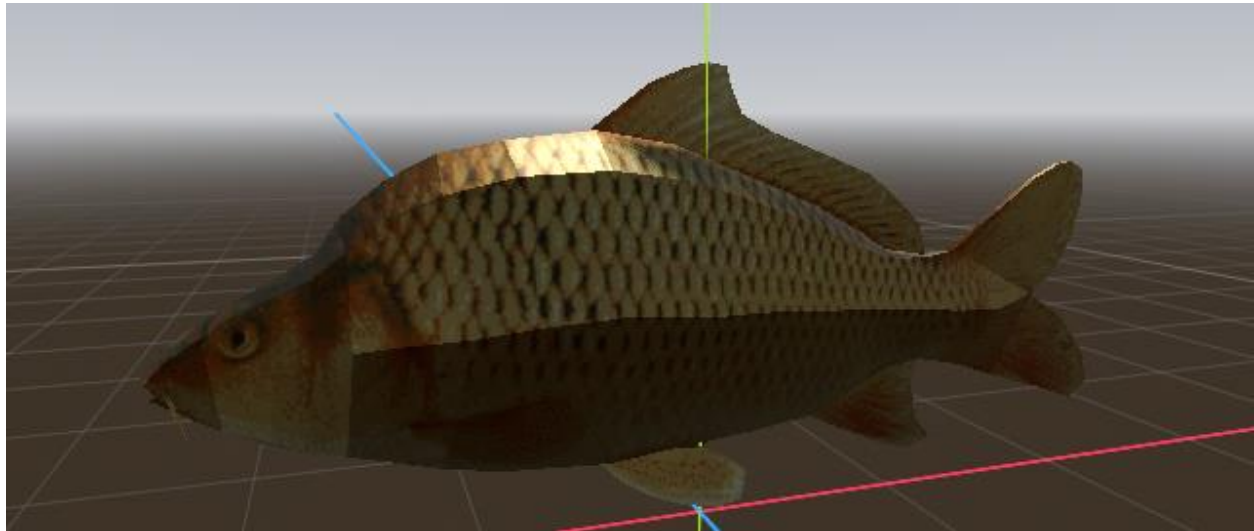
I imported this model into Godot and it had holes in the back fin. I Googled around and asked AI with the screenshot provided why this could be occurring, and culling was something that was listed as a possibility. I thought this was wrong, because there is literally no reason it would cull a random face of the fish for no reason, but to my surprise turning off the culling in Godot did actually fix it.

I wanted to know why it was happening in that specific part in the first place to prevent it from happening again, so I opened Blender back up and I realized that one of the faces was facing the wrong way, which must have happened when I was adjusting the vertices. Exporting the mesh as soon as possible is something I saw mentioned in Blender videos for game dev, because it highlights issues you might not be able to spot in Blender itself. Definitely a good thing to keep in mind for later.

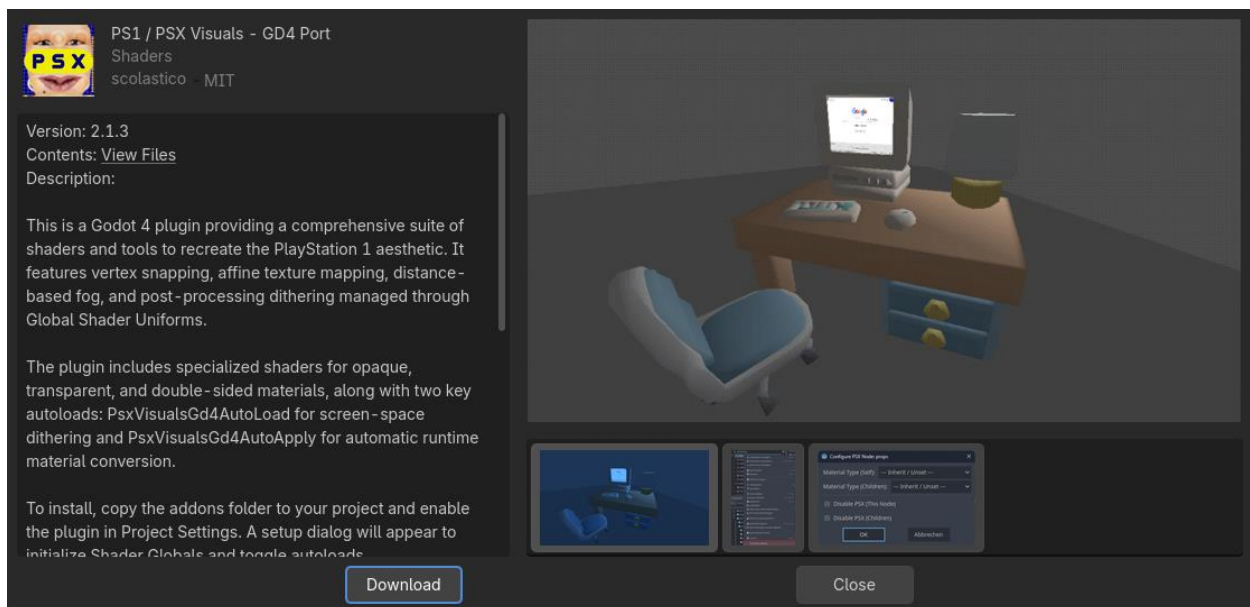


I looked up solutions and found that there is a built in "Flip Normals" function to fix exactly this issue. I re-exported the file back into Godot, this time without the material properties, because I wanted to be able to set the texture manually, so that it would be easy to reuse the same model for different fish.

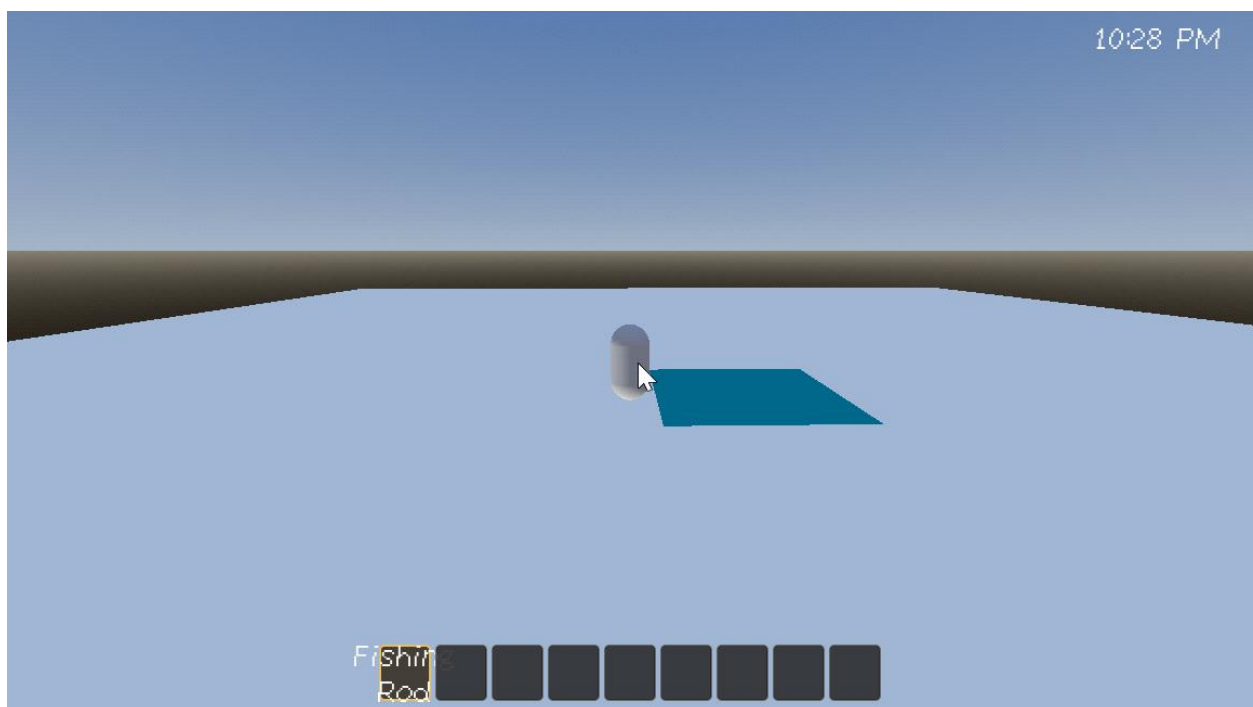
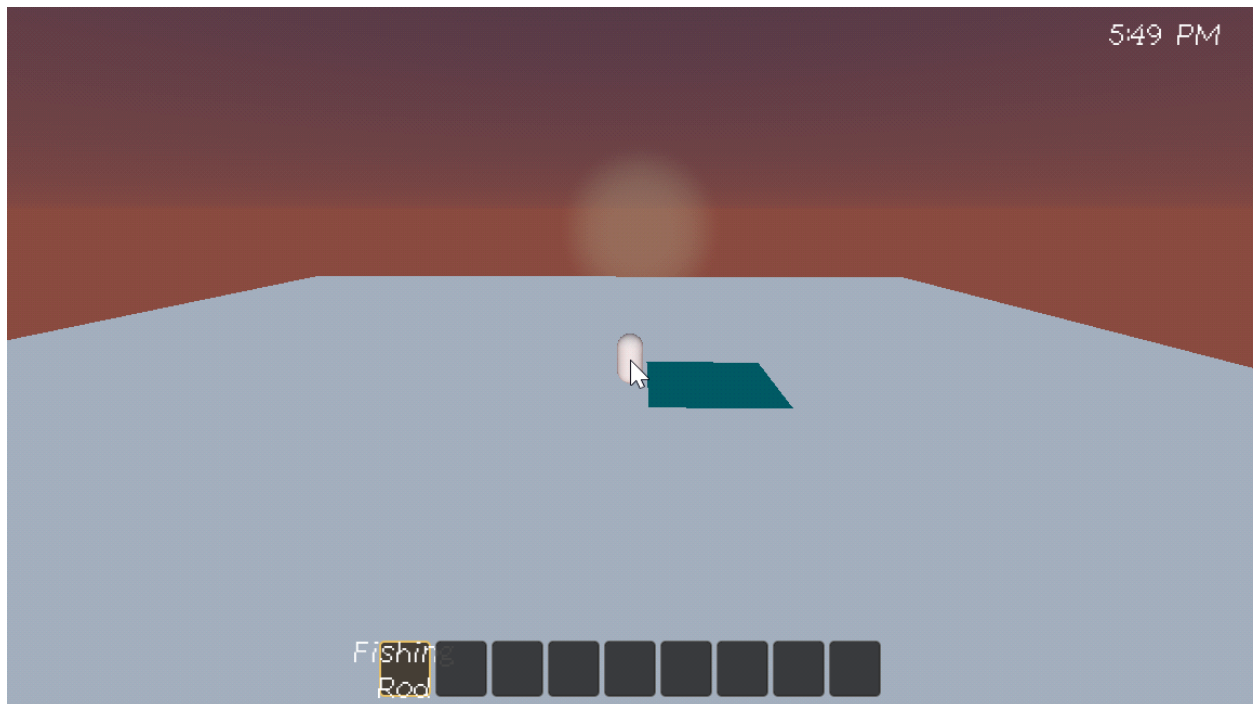
To test this, I quickly made a Golden Carp texture, which after tweaking material properties in Godot looks like this:



With my fish ready, I went back to working in Godot. Before I script the fishing action itself, I thought it would be a good idea to make a foundation for an item system. I started by defining item ids, the ids then are then stored in the inventory array to be displayed. I made a fishing rod item that the player starts with, which is required to fish. I also set up a simple blue collision area as a test for the fishing hole, which for now just checks for the item. Afterwards thought it would be a good idea to look at my shader options, because graphics programming is definitely beyond my current abilities. I looked at the official asset library and found this, which seemed good for my use case so I just decided to go with it.



I set up the shader in accordance with the instructions, tweaking the values until I liked them. I also read the documentation and watched a tutorial on how to set up a World Environment node and a Sky node, which I then scripted to change with my existing time system.



Week 3

At this point I had put a lot of work into the project without really having any idea of how to do the fishing function, and since that's the main gameplay of the prototype, I really needed to start working on that.

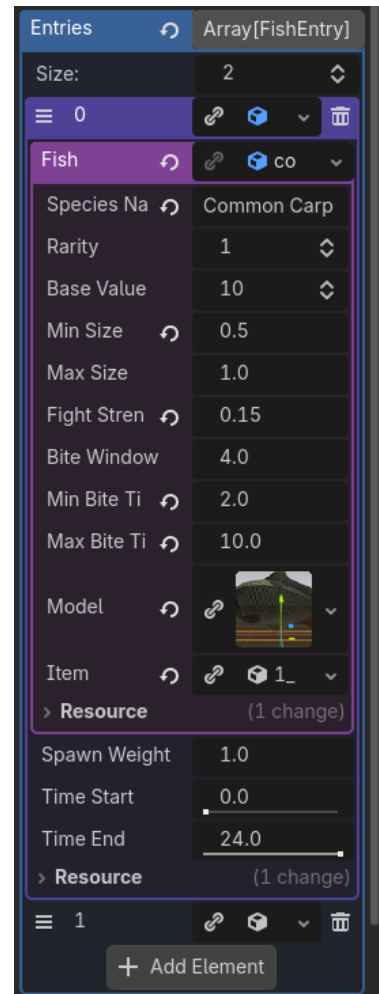
All the systems I built up to this point were essentially prerequisites I set up so the fishing minigame would have something solid to plug into. The signal-based architecture I had been building towards meant I could now design the minigame as its own self-contained scene that just listens for a fish_bite signal, without needing to know anything about how the hole or the clock work. Or that was the idea, in practice it was a lot harder to implement than it sounds.

I started by planning out my structure. I knew I needed data structure for the hole itself, so ideally some way I can put all the fish catchable for a respective hole into an array with their corresponding catch characteristics pre-defined for each fish's data. Then I need detection system for when the player is near the hole, and then a reeling time, a bite, and a fishing minigame after the bite.

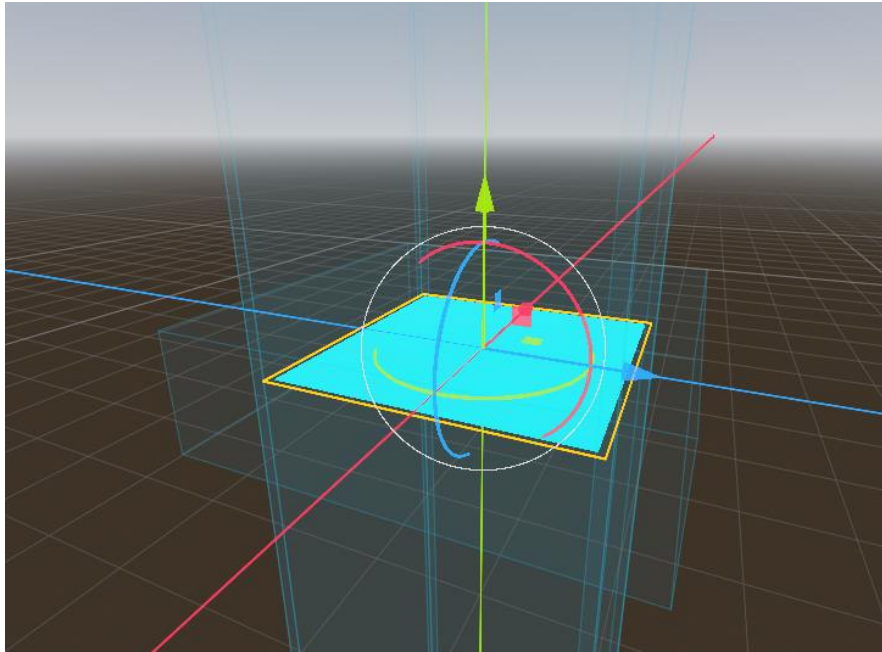
Setting up the fishing hole detection turned out to be more difficult than I expected. In my first version I couldn't figure out why it doesn't work for the longest time, adding a print to every part of the function, unable to identify the reason. I eventually traced it back to the collision shape being almost flat, basically a sheet on the floor, and the player's capsule collider has its center about a meter above the ground so it never actually overlapped, which was definitely a learning moment.

Once I fixed the shape to have actual volume the proximity detection worked, but then I had a second problem with the facing check. I was comparing the player's forward vector against the direction to the hole with a dot product, but the direction vector was pointing mostly downward since the player is standing above the water surface, so the horizontal comparison was basically meaningless. Zeroing out the Y component on both vectors before comparing fixed it, so the check only considers horizontal orientation, which is what actually matters for "are you facing the water."

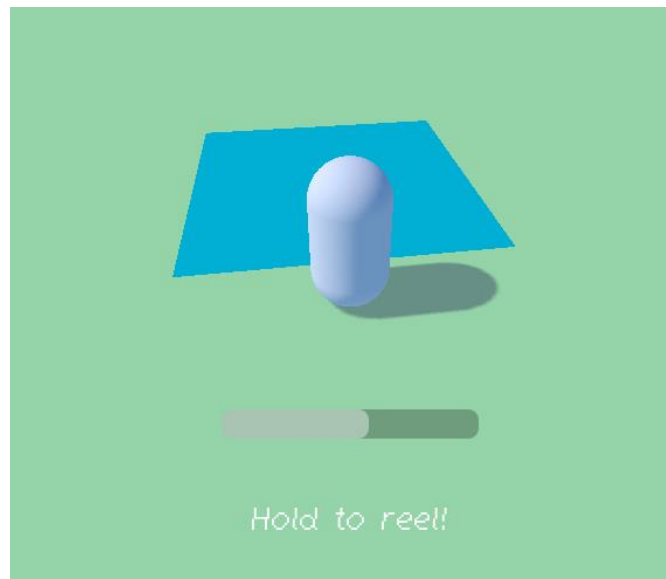
After getting that working I went back and redesigned the whole zone approach, because I'd realized the original rectangular detection shape only really works for perfectly straight, rectangular fishing spots, which basically no real lake or river would be. The solution I found from looking at how others do it in Godot was to use the Area3D collision shape itself as the zone boundary and a raycast from the camera with collide_with_areas enabled for the facing check. The reason we can't just use the raycast immediately is because it's computationally expensive, we want to only use it if it's useful to us, so only when we are next to the hole. That way fishing holes just need a CollisionShape3D that matches the water body, and when I build environments, I can drop one over any irregular lake mesh and it works without touching any code. The debugging process here felt useful in a way a tutorial wouldn't have been, because the whole iterative process is exactly the kind of thing you'd only learn by doing.



The fishing hole detection zone is the square around it, the tall walls are collision blocking walls.



The next thing I needed to do was the fishing mechanic itself. I implemented a state machine for the reel minigame with four states: idle, casting, bite, and reeling. The cast starts on a left-click if you're facing the hole, runs a three second timer to make room for an animation to play, then the bite window opens and you have four seconds to react before the fish swims away. As a temporary mechanic for now, I made it so holding click fills a progress bar while the fish constantly drains it, and hitting 100% catches it. Different fish have different fight strengths so common carp is forgiving and golden carp needs you to hold the button for a longer time. It's obviously not a fun mechanic, but I wanted to try to show the different properties of each fish.



Getting fish to actually appear in inventory took longer than it should have. Reeling to 100% was doing nothing visible, but adding print statements showed the catch was firing. I traced it to two separate silent failures stacked on top of each other: first, the fish resource files had no item field set so the catch check was evaluating null and quietly emitting the wrong signal. Once I fixed that, the slots were still blank because the item resources themselves had no display name or icon, so the slot UI hit the fallback branch

and hiding everything. The fish were in the inventory the whole time, just completely invisible. I temporarily solved it by making the inventory fallback display the item name in the slot, resulting in this:



I also added a "Failed!" message when you click to reel too early during the cast, which pops in with the same bounce animation as the !! bite indicator then fades out automatically. It's a small thing but the game felt confusing without any feedback for that input.

I wanted fish items to display their actual 3D model as the inventory icon rather than a flat texture, which turned into one of the more technically involved things I've done in this project. After looking at docs for the options I had, I found SubViewport, which is specifically used for displaying 3D within UIs.

I read in the docs that you don't want to use viewports for static things, as it's relatively expensive. As a solution, I thought it was a good approach to spin up a SubViewport, render the model into it once, extract the result as an ImageTexture, then free the viewport entirely so it's not sitting in VRAM doing nothing.

Getting the timing right on that was the first roadblock, calling `get_image()` on the same frame you create the viewport just returned a black box because the GPU hasn't actually drawn anything yet. I had to await a `frame_post_draw` signal first before the pixel data exists to read. I also had to tweak the viewport lighting

The last thing I added was a static cache keyed by model path, so if ten inventory slots are all showing the same fish species only the first one goes through the full viewport pipeline and every subsequent one just gets the cached ImageTexture. Static variables are shared across every instance of the class for the lifetime of the process. I think it would be more beneficial to pre-render the fish and just use 2D images for the icons, but I wanted to try my hand at doing it this way, which for now is sufficient while the set of possible fish is small.

This was my result, however, I did not like how the golden fish looked, so I'm thinking later I will either adjust its materials later to be less shiny, or I will simply pre-generate these inventory images. For now, though, I was very happy with the implementation, even though it required a lot of trial and error to get it right.

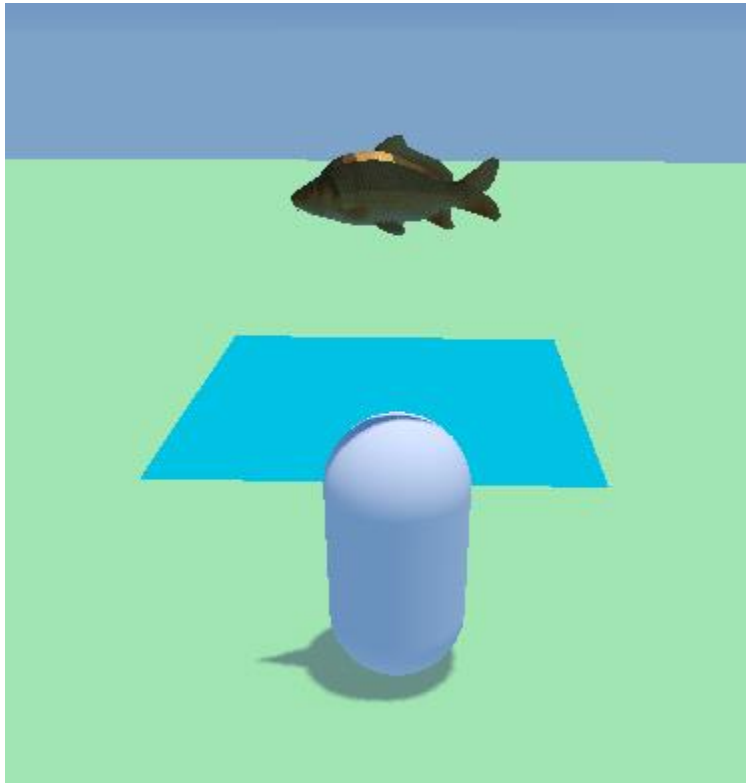


I also went back to my old fish Blender project and scaled it to match Godots scaling, as built-in physics functions assume 1 unit is 1 meter, my fish would currently have about 4.5 meters lengthwise.

I changed the lighting in the inventory viewport, so the golden carp looks better, but making it so reflective just straight up looks weird with such a low polygon count. To achieve reflective surfaces, I might try using a normal map instead later on, but that's definitely not a priority right now.



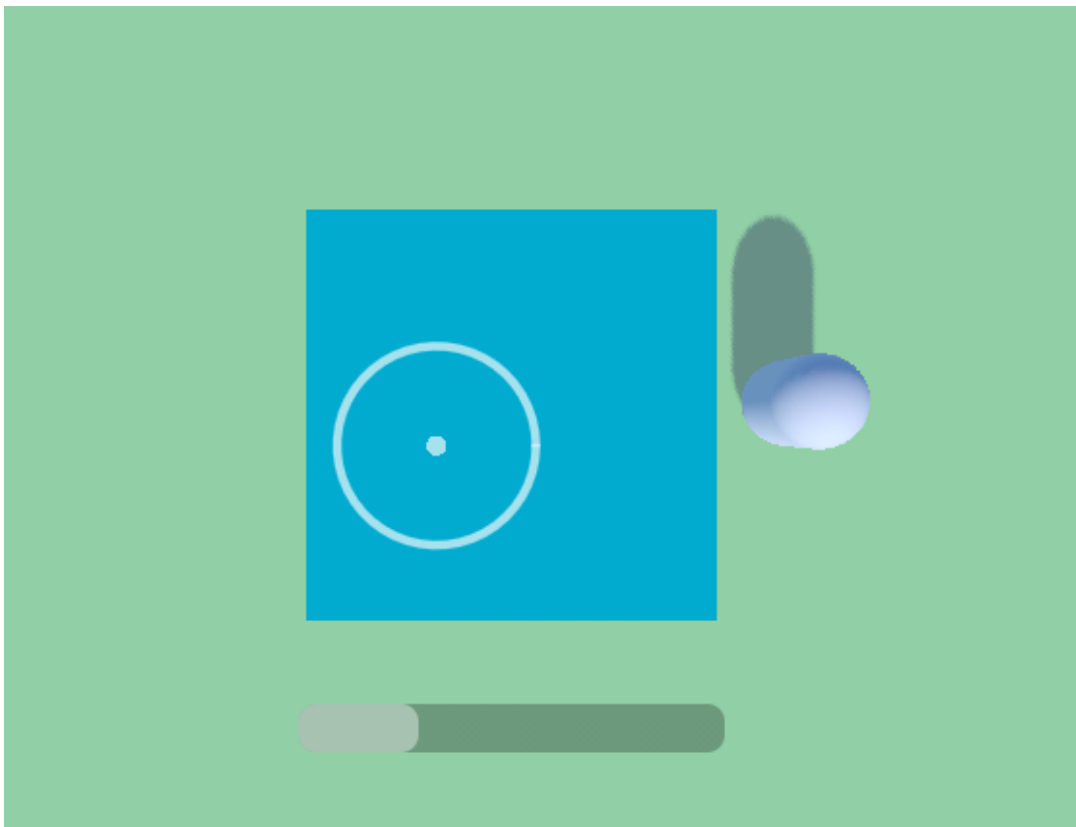
The last big thing this week getting the held item to be shown on the player. For the held item I put the model, position offset, rotation, and scale as fields on the base Item resource rather than on any specific subclass. That way the display node just checks if item.model exists and renders it. Adding a held fishing rod later means filling in four fields in a .tres file in the editor with zero code changes, which should hopefully speed up the process of adding new fish later down the line. The position being a per-item field rather than hardcoded on the node also means each item can sit in a different position in relation to the player.



Week 4

Game development is a lot more difficult than I had anticipated. I already knew it's a lot of work and that it's really hard, but I really did not expect how slow I would be at it. I originally had naive plans for features I would implement by the end of this sprint, but at this pace I'll be happy if I manage to get a working prototype for the idea I had in the game.

I started by making it so instead of holding a button to fill a bar, the camera now lerps from the player view to directly above the fishing hole when the bite triggers, and circles start spawning on the water surface that you have to click before they close in and disappear. I did reuse the bar, though, as hits fill the bar and misses drain it, with spawn rate, lifetime, and drain speed all set per fish type on the resource so tuning difficulty doesn't require touching any code.



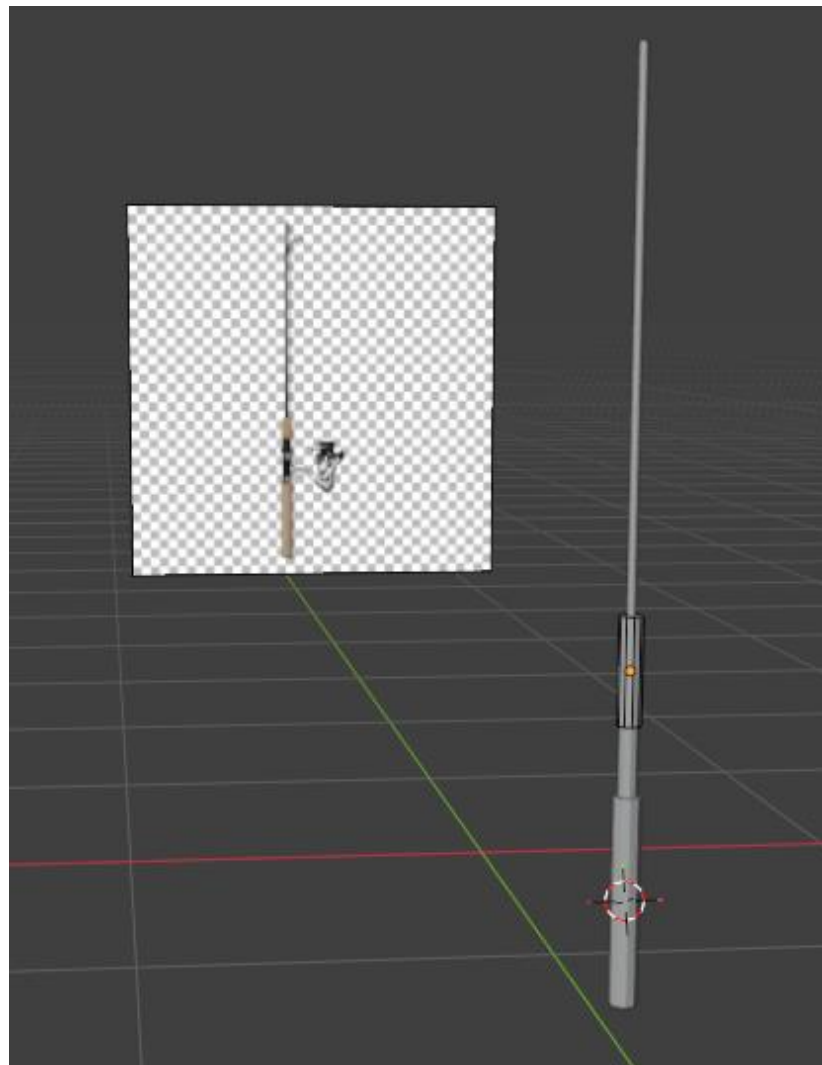
Getting the overhead camera to actually point downward took longer than it should have. I was setting the rotation directly in the scene file but it kept coming out wrong and pointing at the sky, and it turned out the way Godot serialises transforms in .tscn files is easy to get backwards. The fix was to just ignore the rotation on the node entirely and set it in code instead, which is explicit and always does what you expect. I generally found this to be the case with Godot. A lot of things are possible in the editor without code, but often times it's just another place where you can set something and forget, while code is much more organized and readable to me.

I also ran into an issue with the bar never registering completion because the drain was happening in the same frame as the hit check, so a value that reached exactly 100% would get pulled back below before the check ran. Moving the completion check into the hit function rather than the process loop fixed it.

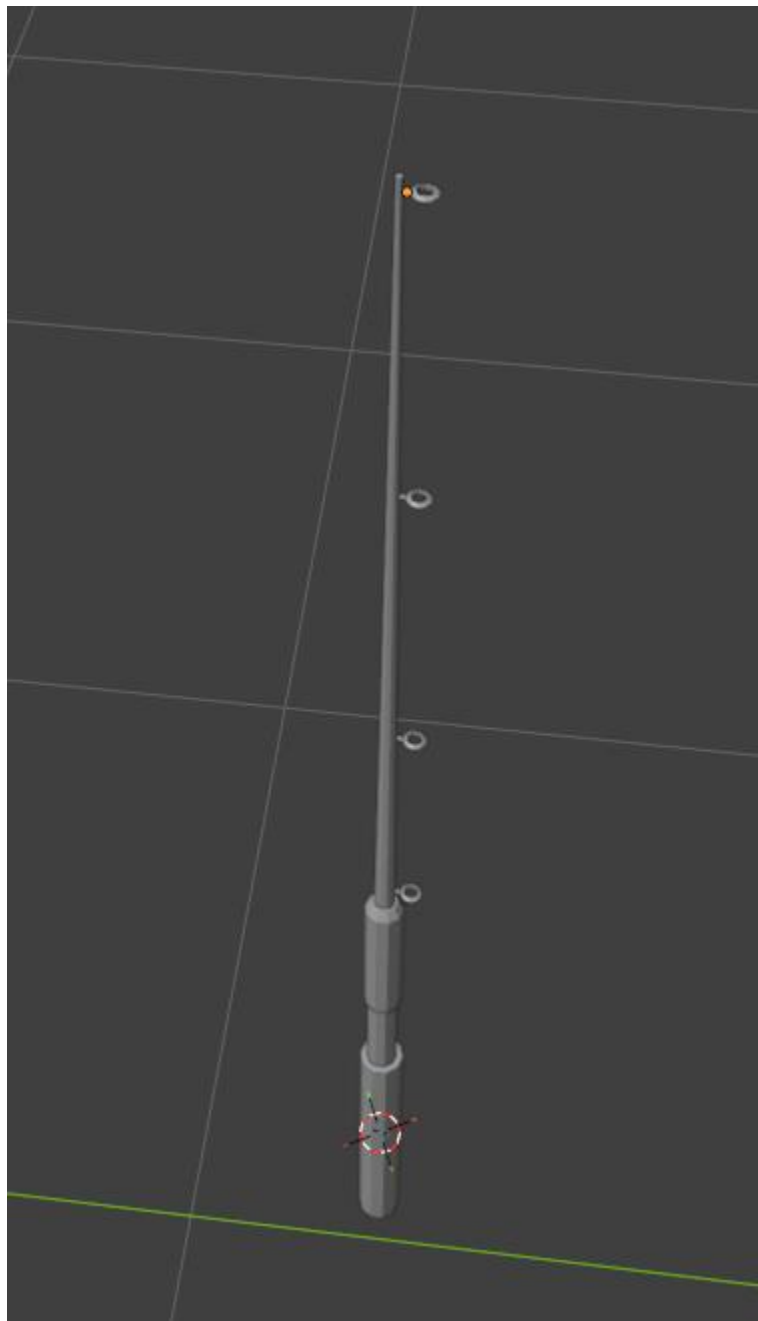
The feedback text for catching and failing went through a few iterations too, mainly around timing, since showing "Caught!" before the camera had finished transitioning back to the player looked wrong. I ended up delaying it by 350ms with a timer to match the fade transition, which was a cleaner solution than trying to await the tween. I also added a bouncy animation on enter/exit using some vector scaling transforms and tweens.

The reason I like this mechanic so much and it's been the whole reason why I'm making this game is because it allows for a lot of design possibilities, making for harder or easier fish or fish with interesting patterns. I don't know yet how I will animate the fishing action itself, as this is very much not how fishing works in real life, but that's a problem for future me.

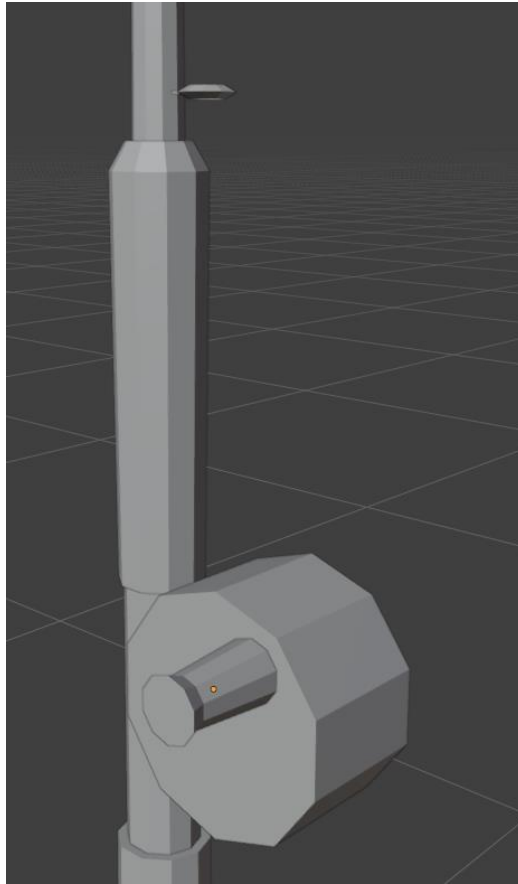
With that done, I felt as though it would be a good idea to try modelling a fishing rod item to not have to use any more placeholders. I started the same way as before, with a reference image, but this time I also thought about the scale. I did some research and found that fishing rods are normally between 1.8-2m long, including the handle, so that's the size I was aiming for. I started with a relatively low poly cylinder that I matched with the handle. I started with a cylinder for the handle, followed by a few more differently sized cylinders for the rest of the rod.



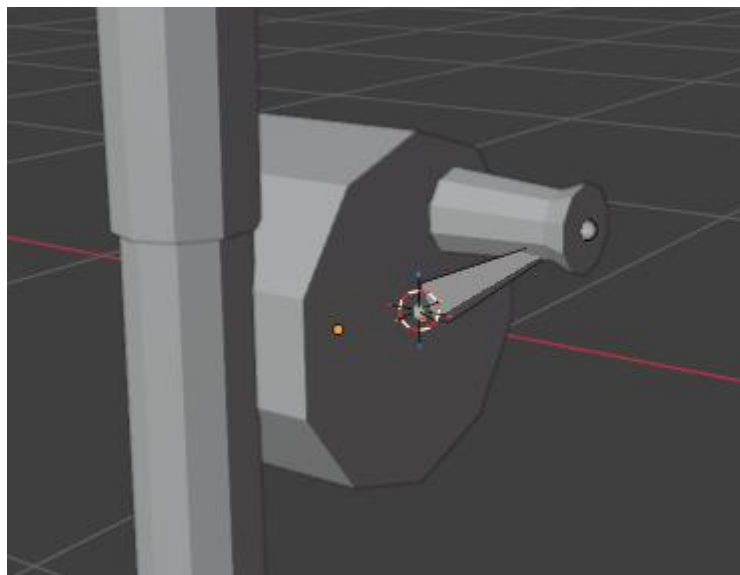
I continued by smoothing out the handles and adding the loops, for which I used really small tori.



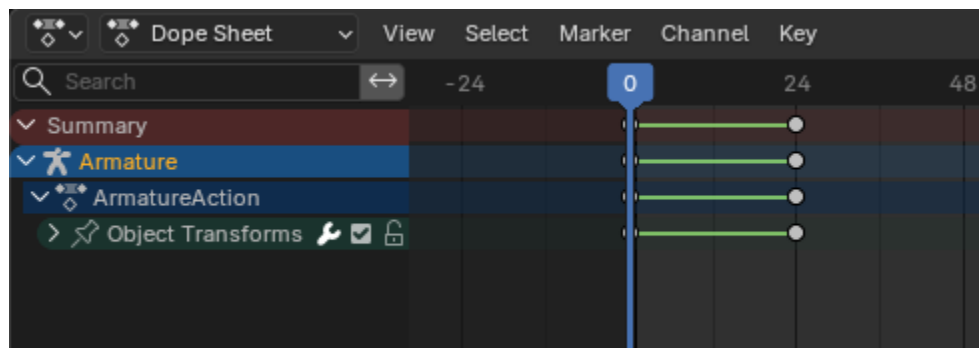
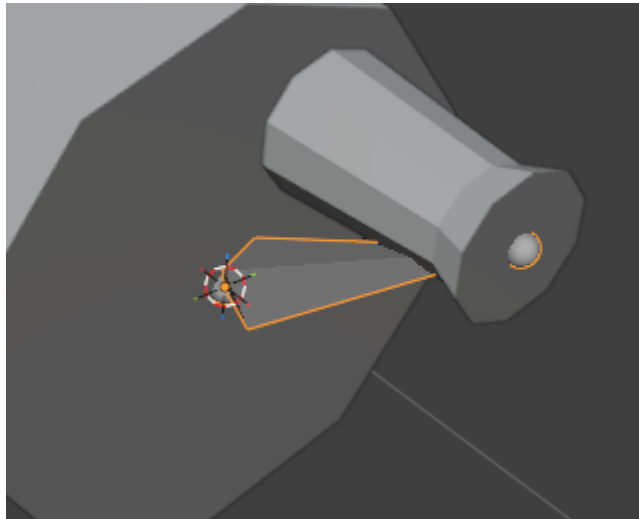
All that was left was the crank, which was fairly straightforward:



At this point I realized that I need to rig it in order to animate spinning the crank, which I had no idea how to do. I improvised and used the 3D cursor to line up the armature, which seems to be the correct approach from the few YouTube videos I've skimmed through afterwards.

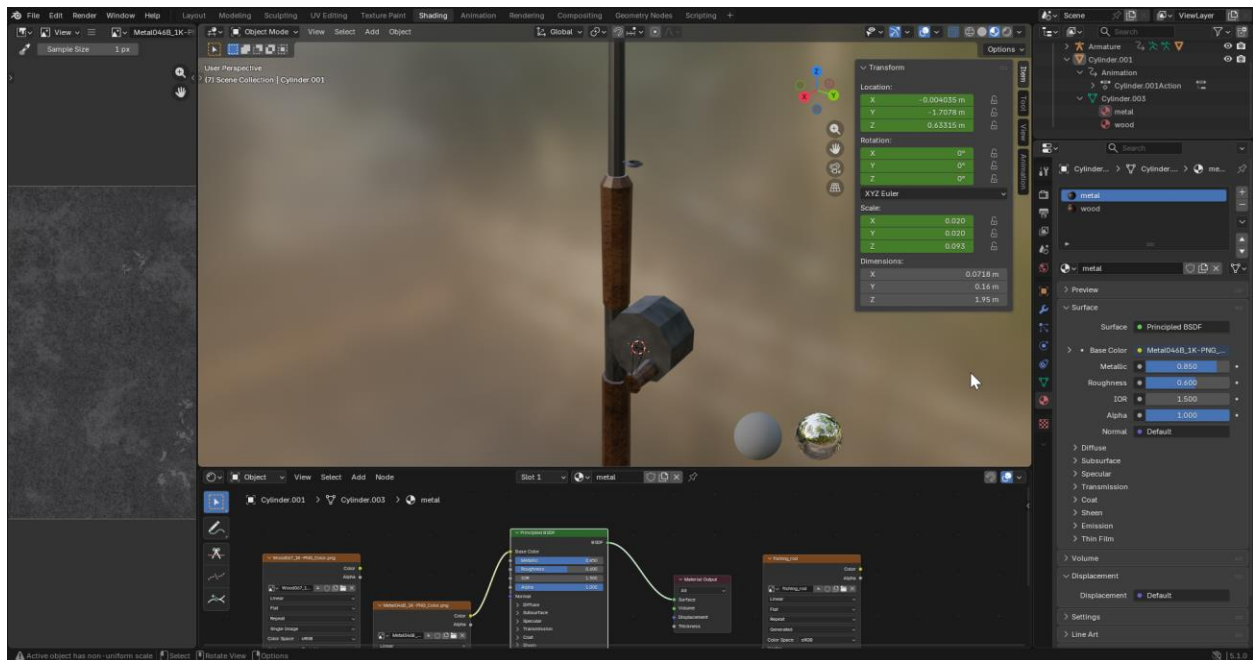


After an excruciating 3 hours of trying to understand why my handle isn't spinning around the crank properly, I found out that the origin of the crank actually matters and that it should be set to the pivot point.

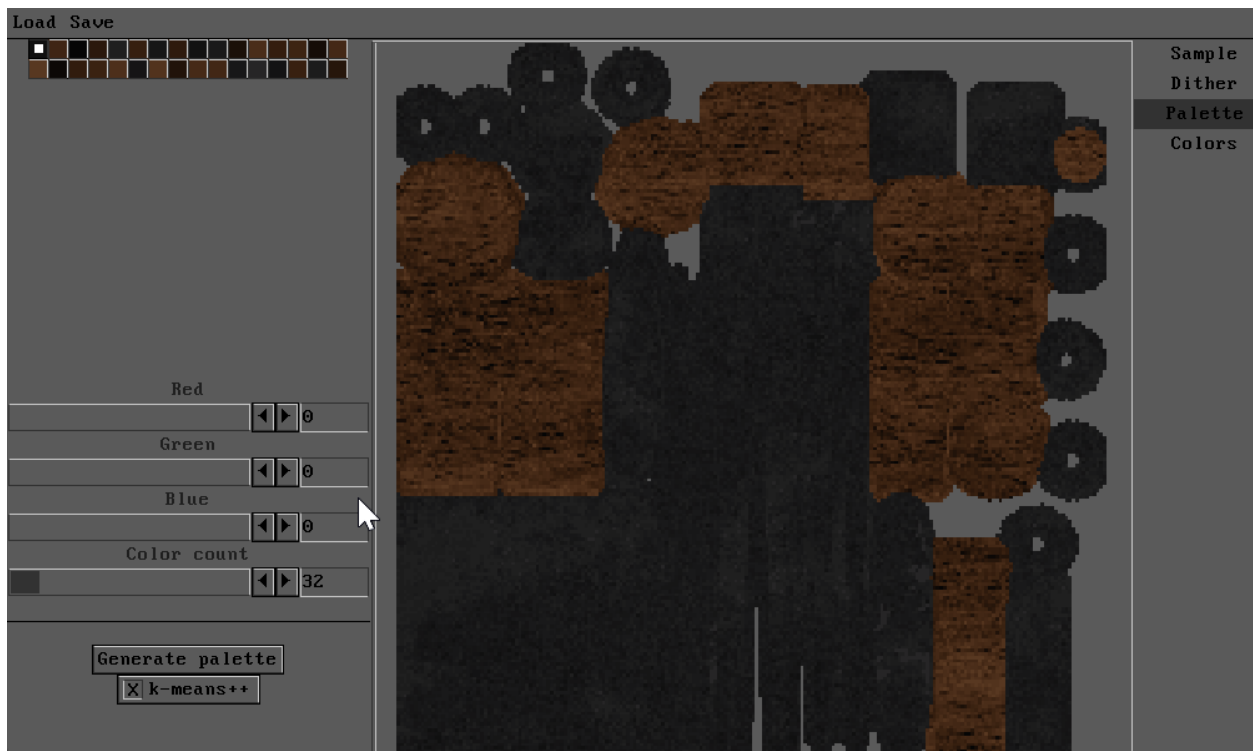


Adding the animation frames itself was easy, I just made one at X rotation 0, and the other at X rotation - 359, which should hopefully loop. I also changed the interpolation to be linear since I want it to be loopable.

I also told myself I won't just grab the texture from the rod from the reference image and I'll try my hand at painting textures to at least know how it works and how hard it is. I followed a tutorial for this, but now I'll be able to do it myself. I went onto ambientcg.com where I downloaded a metallic material and a wood material, then applied it to various parts of the rod.



This looked nice, however, I wanted to retain the same pixelated look for all our assets, so I baked these textures into a clean texture file, then exported them, used Img2Pixel again, and imported them back in. It took me a few attempts because I forgot to change bake mode to diffuse, but I figured it out eventually, the end result being this:



Afterwards, I tried exporting it from Blender, but the position of the crank kept getting moved in the export. After an arduous hour of Googling, I found that the issue is from the animation position being saved in world space, so after animating it's not possible to move something within the scene, which shouldn't matter for the sake of Godot exporting, but I still wanted to know why it was happening so that I'd understand it. transforms on the crank itself. Doing CTRL+A into "All Transforms" solved my problem, and importing it into Godot left me with this, which I was very pleased with:



At this point, I have run out of time, but I was still happy with my work overall. I was really surprised with just how hard game development is, especially since I haven't really animated anything properly. Overall, I'd say I learned the most during this sprint than any other sprint in the minor, and I definitely picked up the most diverse set of skills as a result of it. I'm happy I found something that I really enjoy, and I'll definitely keep chipping away at this project in the near future.

[Kanban](#)